Part B: Design Normalized Database

Amanda Gale & Priamos Koumas

Fall 2026

Question 1: For the relation represented by all of the columns in the CSV file and the table in the SQLite database, define all functional dependencies and list them.

Restaurants

$$\texttt{RestaurantID} \rightarrow \texttt{RestaurantName}, \texttt{City}, \texttt{State}, \texttt{hasTableService}$$

Servers

$$\texttt{ServerEmpID} \rightarrow \texttt{ServerName}, \texttt{StartDateHired}, \texttt{EndDateHired}, \texttt{HourlyRate}$$
$$\texttt{ServerEmpID} \rightarrow \texttt{BirthDate}, \texttt{SSN}, \texttt{rID}$$
$$\texttt{SSN} \rightarrow \texttt{ServerEmpID}$$

Customers

$$\texttt{CustomerEmail} \rightarrow \texttt{CustomerName}, \texttt{CustomerPhone}, \texttt{LoyaltyMember}$$

Visits

Each row of the CSV is one visit, which determines the rest of the row.

$\texttt{Visit} \rightarrow \texttt{RestaurantID}, \texttt{ServerEmpID}, \texttt{CustomerEmail}, \texttt{VisitDate}, \texttt{VisitTime}$
$\texttt{Visit} \rightarrow \texttt{MealType}, \texttt{PartySize}, \texttt{WaitTime}$
$\texttt{Visit} \rightarrow \texttt{FoodBill}, \texttt{TipAmount}, \texttt{FoodDiscountPercentage}, \texttt{PaymentMethod}, \texttt{orderedAlcohol}, \texttt{AlcoholBill}$

Question 2: Using the functional dependencies and the rules of normalization, decompose the relational from the CSV and the SQLite database into several relations that all satisfy 3NF; give the relations reasonable names.

Original relational form:

- RestaurantVisits(RestaurantID, RestaurantName, City, State, hasTableService, ServerEmpID, ServerName, StartDateHired, EndDateHired, HourlyRate, BirthDate, SSN, CustomerName, CustomerPhone, CustomerEmail, LoyaltyMember, VisitDate, VisitTime, MealType, PartySize, WaitTime, FoodBill, TipAmount, FoodDiscountPercentage, PaymentMethod, orderedAlcohol, AlcoholBill)

Relations in 3NF:

- restaurants(rid, restaurantname, city, state, hastableservice): RestaurantID -> RestaurantName, City, State, hasTableService
- servers(empid, servername, startdatehired, enddatehired, hourlyrate, birthdate, ssn, rid): ServerEmpID -> ServerName, StartDateHired, EndDateHired, HourlyRate, BirthDate, SSN, rID
- customers(cid, customername, customerphone, customeremail, loyaltymember): CustomerEmail -> CustomerName, CustomerPhone, LoyaltyMember
- bill(billid, foodbill, tipamount, fooddiscountpercentage, pmid, orderedalcohol, alcoholbill): Visit -> FoodBill, TipAmount, FoodDiscountPercentage, PaymentMethod, orderedAlcohol, AlcoholBill
- paymentmethods(pmid, paymentmethod): lookup table for Visit -> PaymentMethod
- mealtypes(mtid, mealtype): lookup table for Visit -> MealType
- visits(visitid, customerid, restaurantid, serverempid, billid, mtid, visitdate, visittime, partysize, waittime): Visit -> VisitDate, VisitTime, PartySize, WaitTime

Definitions

1NF:

1. All attributes must contain atomic (indivisible) values. This means that each attribute in the table can only hold a single value.
2. The relation must have no repeating groups of attributes or tuples. Each row must represent a single, distinct entity or concept.

2NF:

1. It is in 1NF.
2. All non-prime attributes (attributes that are not part of the primary key or any candidate key) are fully functionally dependent on the entire primary key, meaning there are no partial dependencies.

3NF:

1. It is in 2NF.
2. No transitive dependencies exist. A transitive dependency occurs when a non-prime attribute is functionally dependent on another non-prime attribute. A non-prime attribute is an attribute that is not part of any candidate key of a relation.

The original relation is in 1NF and 2NF because it has no repeating groups and the non-prime attributes depend on the entire primary key. However, values like RestaurantID repeat across many rows with their details, since non-prime attributes depend on other non-prime attributes (RestaurantID -> RestaurantName). By breaking it down into multiple tables, including lookup tables, no transitive dependencies exist, allowing for 3NF. Three new IDs were added when creating the new relations (billid, visitid, and cid), since no column in the original data uniquely identifies a bill, visit, or customer. Orderedalcohol is kept in bill because the alcohol bill can still be 0, even if alcohol is ordered, therefore AlcoholBill does not determine orderedAlcohol with this specific business logic.

Entity Relationship Diagram

